

# THE LANGSELECT PACKAGE

vo.2.1      langselect

Generate different language versions from a common source

Matthias WERNER<sup>1</sup>

<https://github.com/tuc-osg/osglecture>

This package supports the generation of documents in two or three language versions from a common source document. A corresponding user macro is generated, and there are several methods for determining the target language.

The package is part of the `osglecture` bundle, is integrated into the `osglecture` class and supported by the build script `ol1m`, but can also be used independently.

## Table of Contents

|          |                                  |          |          |                                        |           |
|----------|----------------------------------|----------|----------|----------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>              | <b>1</b> | <b>3</b> | <b>User Commands</b>                   | <b>6</b>  |
| 1.1      | TL;DR . . . . .                  | 1        | 3.1      | Automatic Language Commands            | 6         |
| 1.2      | Motivation . . . . .             | 2        | 3.2      | Starred Variant . . . . .              | 7         |
| 1.3      | Alternatives . . . . .           | 2        | 3.3      | Individual Language Commands           | 8         |
| 1.4      | Languages . . . . .              | 3        | <b>4</b> | <b>Interaction with Other Packages</b> | <b>8</b>  |
|          |                                  |          | 4.1      | Babel/Polyglossia . . . . .            | 8         |
|          |                                  |          | 4.2      | Csquotes . . . . .                     | 9         |
| <b>2</b> | <b>Usage</b>                     | <b>4</b> | <b>5</b> | <b>FAQ</b>                             | <b>9</b>  |
| 2.1      | Selection of the Target Language | 4        | <b>6</b> | <b>Implementation</b>                  | <b>10</b> |
| 2.2      | Options . . . . .                | 5        |          |                                        |           |

## 1 Introduction

### 1.1 TL;DR

Assume, you have a text with an English and a German version whilst you want to use a common source.

---

<sup>1</sup>. [matthias.werner@informatik.tu-chemnitz.de](mailto:matthias.werner@informatik.tu-chemnitz.de)

1. Include `\usepackage[languages={en,de}]{langselect}`
2. Use in the source `\lende{ $\langle text_{en} \rangle$ }{ $\langle text_{de} \rangle$ }`.  $\langle text_{en} \rangle$  will appear in the English version, whereas  $\langle text_{de} \rangle$  will appear in the German one.
  - Write text that appears in both versions outside the macro or use the star version so that you only have to write common data in the body text once, see Section 3.2.
3. Translate the document with `o` get the English version and with `langselect/language/after` for the German version.
 

```
> latex "\def\olsTargetLanguage{en}\include{<document>}"
> latex "\def\olsTargetLanguage{de}\include{<document>}"
```

  - There are several other ways to determine the target language, including via the job name and via `\DocumentMetadata`.

## 1.2 Motivation

Sometimes, multiple language versions of the same document are required. In the application area of the `osglecture` bundle, these are, for example, the lecture notes, lecture slides, and handouts for a course. It is often difficult to keep multiple parallel language versions consistent. This task is somewhat easier if all versions are contained in a common source document.

This package supports the generation of such language versions from a common  $\text{\LaTeX}$  source document by providing commands for language variants and mechanisms for selecting the target language.

## 1.3 Alternatives

There are a number of alternatives to `langselect`.

**Ad hoc commands** It is relatively easy to create ad hoc solutions, such as the following

```

1      \newif\ifenglish
2      \englishtrue
3      \ifenglish
4          Welcome!
5      \else
6          ¡Bienvenido!
7      \fi
8
```

In fact, `langselect` basically follows this approach, but automates the creation of language commands, simplifies the sharing of non-language-dependent elements such as formulas, and enables the selection of the target language by different approaches.

**multilang** The sophisticated package [multilang](#)<sup>2</sup> by Richard GREWE creates language versions of given commands, whereby the active language is taken directly from [Babel](#) or [Polyglossia](#). [langselect](#) has a similar goal, but a different (from the package author’s point of view: more practical) interface. However, if you don’t like the approach of [langselect](#) you should definitely take a look at [multilang](#).

**comment** Victor EIJKHOUT wrote the package [comment](#),<sup>3</sup> which makes it easy to output only certain sections of the document. This can also be used very well for language selection.

**bicaption/translations/translator/xt\_cpts** The packages are intended for programmers of L<sup>A</sup>T<sub>E</sub>X packages to internationalize fixed text elements such as the headings of tables of contents. These packages are only of limited use for document authors. [langselect](#)/[language/afterbicaption](#)<sup>4</sup>/[translations](#)<sup>5</sup>/[translator](#)<sup>6</sup>/[xt\\_cpts](#)<sup>7</sup>

## 1.4 Languages

The concept of [langselect](#) involves the use of languages at different levels with different effects. To avoid confusion, we want to clearly define the terms here.

1. The set of *possible* target languages for the document. A document that uses [langselect](#) typically contains text sections in different target languages. The possible target languages are set via a package option, see Section . In the remaining part of this documentation, we refer to the *possible* target languages as *selectable languages*, since one of these languages is selected for the L<sup>A</sup>T<sub>E</sub>X translation. [langselect/language/after2.2](#)
2. The actual target language used in a LaTeX translation. This must be one of the languages from 1. Typically, the translation process is performed separately for each of the actual target languages. There are several methods for selecting the actual target language; see section 2.1.
3. The language in which text fragments or individual words are actually written, what is considered, e. g. for hyphenation. This is the concept of language used by packages such as [Babel](#) or [Polyglossia](#).<sup>8</sup> Please note that a multilingual document from the perspective of [langselect](#) can be a monolingual document from the perspective of [Babel](#). Frequently, it is the other way around, and more languages than the selectable languages are used: For example, a document with the selectable languages German and English may contain Latin words or sentences in the text. This selection is made using the usual mechanisms such as `\selectlanguage`, `\foreignlanguage` or `\text{language}`. When [Babel](#) or [Polyglossia](#)

---

2. on CTAN as [multilang](#): <http://mirrors.ctan.org/macros/latex/contrib/multilang/>

3. on CTAN as [comment](#): <http://mirrors.ctan.org/macros/latex/contrib/comment/>

4. on CTAN as [bicaption](#): <http://mirrors.ctan.org/macros/latex/contrib/bicaption/>

5. on CTAN as [translations](#): <http://mirrors.ctan.org/macros/latex/contrib/translations/>

6. on CTAN as [translator](#): <http://mirrors.ctan.org/macros/latex/contrib/translator/>

7. on CTAN as [xt\\_cpts](#): [http://mirrors.ctan.org/macros/latex/contrib/xtcpts/xt\\_cpts/](http://mirrors.ctan.org/macros/latex/contrib/xtcpts/xt_cpts/)

8. However, in these packages, “language” also refers to several different and, in some cases, independent concepts such as hyphenation patterns, fonts to be used, etc.

is loaded, which can also be done directly via `langselect` if desired, the target language is selected as the active language.

Languages are designated in `langselect` using ISO 639-1 codes (“en”, “fr”, “de” ...). If a specific dialect or variant is desired for language processing at the Babel/Polyglossia level, mapping can be performed.

## 2 Usage

The package is loaded in the usual way:

```
\usepackage[options]{langselect}
```

Since the target language can be selected both via options and in other ways, these different methods are described here first. Further options, such as setting the selectable languages, are discussed in Section `langselect/language/after2.2`

### 2.1 Selection of the Target Language

There are various ways to determine a target language for a LaTeX translation run. The order in the following list also reflects the order of evaluation. Once a target language has been determined, further evaluation is canceled.

1. Definition of a macro `\olsTargetLanguage` *before* loading the package. This makes it possible, for example, to use

```
> latexmk -usepretex="\def\olsTargetLanguage{fr}" document.tex
```

to set the target language French for the translation run.

2. Setting of the packet option `Various values are possible here``langselect/language/after``targetlang`:

```
targetlang = <ISO code>
```

The target language is specified directly by the code.

```
targetlang = {job=n}
```

The target language is determined from the *n*th element of the job name. An element is a part of the job name separated by “-” (minus), see the documentation for the package `varsfromjobname`,<sup>9</sup> which is loaded for this purpose. For example, with `targetlang = {job=2}` one can set Russian as target language using

```
> latex -jobname doc-ru doc.tex
```

```
targetlang = babel|polyglossia
```

To use this key value, Babel or Polyglossia must be loaded *before* `langselect`. The target language is then set to the (main) language set by the language packet.<sup>10</sup>

9. on CTAN as `varsfromjobname`: <http://mirrors.ctan.org/macros/latex/contrib/varsfromjobname/>

10. Both values have exactly the same effect: Since Babel and Polyglossia offer the same interface, it doesn’t matter which of the two packages is loaded. Thus, `targetlang = polyglossia` will also find Babel.

`targetlang = meta`

Uses the language specified in `\DocumentMetadata` as the target language.

3. If neither `\olsTargetLanguage` is defined nor the option `targetlang` is set, `langselect` attempts to determine the target language on its own by first trying the option `targetlang = babel` and then (if no language package was found) the option `targetlang = meta`.
4. If everything else fails, English is set as the target language.

## 2.2 Options

The following options are available:

`languages = {⟨list of selectable languages⟩}` (required)

Provides two or three selectable languages. The order matters for the automatic generation of the language commands (cf. Section . `langselect/language/after3`)

`targetlang = ⟨language⟩|{job=⟨n⟩}|babel|polyglossia|meta`

See Section 2.1

`prefix = {⟨prefix⟩}` Default: 1

Sets a prefix for multilingual commands. An empty prefix can also be set.

`auto = true|false` Default: true

The actual language commands are created automatically, see Section . If `auto = false` is set, only the commands for generating the language commands are provided, not the language commands themselves. `langselect/language/after3`

`trim = true|false` Default: true

Removes whitespace at the beginning and end of language macro arguments.

`load babel = {⟨Babel options⟩}`

`load polyglossia` Polyglossia options Loads either Babel or Polyglossia so that the target language of `langselect` is set as the main language. The options passed here are forwarded to the respective language package. If one of these options is used, `targetlang = babel` or `targetlang = polyglossia` must not be set, and `langselect` must be loaded before the respective language package.

`map = {⟨language⟩=⟨variant⟩,...}`

Defines a mapping of languages to a language variant. When `langselect` interacts with other packages, the specific language variant is passed instead of the ISO code.

For example, if you load Babel from `langselect` and your document explicitly uses *British* English and *Swiss* German, use the following code to ensure that Babel receives the correct options.

```

1 \documentclass{article}
2 \usepackage[
3   languages={en,de},
4   load=babel,
5   map={en=british,de=nswissgerman}
6 ]
7 \begin{document}
8   \title{\lende{Hallo}{Grüezi}}
9

```

`unified shorthands = true|false`

Default: true

Defines cross-language shorthands to support `csquotes`

## 3 User Commands

### 3.1 Automatic Language Commands

If you have not set the option `auto = false`, a multilanguage command will be created. Its name is determined by the list of selection languages. Depending on whether you have specified two or more selection languages, the name is

- $\langle \text{prefix} \rangle \langle \text{language}_1 \rangle \langle \text{language}_2 \rangle \{ \langle \text{text}_1 \rangle \} \{ \langle \text{text}_2 \rangle \}$

or

- $\langle \text{prefix} \rangle \langle \text{language}_1 \rangle \langle \text{language}_2 \rangle \langle \text{language}_3 \rangle \{ \langle \text{text}_1 \rangle \} \{ \langle \text{text}_2 \rangle \} \{ \langle \text{text}_3 \rangle \}$

For example, for this documentation, the package was loaded with the following options:

```

1 \usepackage[
2   languages={de,en},
3   targetlang={job=2}
4 ]{langselect}

```

This makes the command `\ldeen` available.

If the target language “de” is now selected (here via the job name, but there are also other options, see Section , only  $\langle \text{Text}_1 \rangle$  is transferred to the PDF document and  $\langle \text{Text}_2 \rangle$  is discarded. With the target language “de”, it is the other way around. [langselect/language/after2.1](#)

#### Attention!

The automatic generation of language commands can sometimes cause problems. For example, Abkhaz (code: “ab”) as the first language and Greek (Code: “el”) as the second

language results in the automatic name `\label`, which leads to error messages. In such cases, use the features described in section 3.3.

### 3.2 Starred Variant

A common problem with multiple language versions is the consistency of non-language-dependent data, especially if this data changes frequently (think, for example, of timetables).

For such cases, `langselect` provides a starred variant of the language commands. In this starred variant, the text arguments can contain placeholders of the form “@<n>” can be used, where <n> represents a value between 1 and 9.

Additional arguments must then be specified after the command, namely as many as correspond to the highest <n> used. The starred variant replaces the placeholders with the corresponding arguments.

The star variant of the language macros is “fragile”, please do not use them in a mobile context, e.g., within an argument of `\section`.

```

1 \documentclass{article}
2 \usepackage[languages={en,fr},
3   targetlang=fr]{langselect}
4
5 \begin{document}
6 \lenfr*{
7   The formula @1 generally refers to the Pythagorean theorem.
8 }{
9   La formule @1 fait généralement référence au théorème de
10  Pythagore.
11 }{ $a^2+b^2=c^2$ }
12
13 \lenfr*{
14   Calculate the length of the third side if one side is @1 units long
15   and the hypotenuse is @2 units long.
16 }{
17   Calculez la longueur du troisième
18   côté si un côté mesure @1 unités et l'hypoténuse @2 unités.
19 }{ $a$ }{ $b$ }
20 \end{document}
```

La formule ait généralement référence au théorème de Pythagore. langselect/language  
 $b^2 = c^2$   
 Calculez la longueur du troisième côté si un côté mesure nités et l'hypoténuse  
 langselect/language/after unités. 45

### 3.3 Individual Language Commands

If you do not like the generated names of the language commands or if they even lead to collisions, you can change them. The easiest way is to change the package option `prefix` so that, for example, with `prefix = mv`, the language command for the German/English combination is called `\mvdeen`.

If this is not sufficient, the following two commands give you complete control over the naming.

```
\olsMakeBilingualMacro[⟨prefix⟩]{⟨lang1⟩}{⟨lang2⟩}[⟨base name⟩]
```

```
\olsMakeTrilingualMacro[⟨prefix⟩]{⟨lang1⟩}{⟨lang2⟩}{⟨lang3⟩}[⟨base name⟩]
```

These commands create a bilingual or trilingual language command for the specified languages. If `⟨base name⟩` is specified, the name of the language command is `\⟨prefix⟩⟨base name⟩`. If the `⟨prefix⟩` argument is used, it determines the prefix; otherwise, the package options apply.

## 4 Interaction with Other Packages

### 4.1 Babel/Polyglossia

The `langselect` package is initially independent of language packages such as Babel or Polyglossia. It can be used without or together with these packages. However, if you take the trouble to create a multilingual document, you will usually also want the correct hyphenation rules, font settings, date formats, etc. to be used for the respective language.<sup>11</sup>

Please note that as long as you do not use a different language *within* a text (perhaps the language of the other language version), your document is, from the perspective of Babel, a *monolingual* document. There is a risk that you will invoke Babel in the usual way with the various languages of your language version but, for determining the target language, do not use the option `targetlang = babel`. As a result, the target language may not match Babel’s active language, and you may end up with incorrect hyphenation and similar errors.

To avoid such problems, `langselect` offers some options:

1. As mentioned above, you can use the package option `targetlang = babel`. Then the target language is determined by the main language of Babel. However, you then have to forego the option of controlling the target language “from outside” (i.e., with `\olsTargetLanguage` or via the job name).
2. In turn, `langselect` loads Babel or Polyglossia using the options `load babel/load polyglossia` and sets the target language as the main language. Additional options for the language packages are passed as option arguments.
3. Finally, the hooks `langselect/language/before` and `langselect/language/after` can be used to insert any code before or after the arguments of the language macro are output, e.g., `\foreignlanguage` may be used here. The before hook has the (mapped) target language as a parameter.

---

11. In the following, I will refer exclusively to Babel. However, what has been said also applies accordingly to Polyglossia.



## 4.2 Csquotes

Different languages have different conventions for punctuation. This is particularly problematic with quotation marks, as these should follow the target language, but may also appear outside language macros or in arguments of the asterisk variant. The `csquotes`<sup>12</sup> package offers a solution for this by providing `\enquote` and other macros that follow the quotation convention of the current language.

For added convenience, `langselect` supports the use of correct quotation marks with language-independent shorthands: If the option `unified shorthands` is true, `csquotes` is loaded and the shorthands `"`` and `"'` are set to `\openautoquote` and `\closeautoquote`, respectively.

## 5 FAQ

- **Is `langselect` still experimental?**

Absolutely. Although the author uses it on a daily basis, it cannot be ruled out that there may be undesirable interactions with other packages. The version number indicates this.

- **What does the “o” in the `\ols` prefix stand for?**

The package belongs to the `oslecture` bundle, from which the “o” is inherited.

- **Are there any application examples that go beyond a toy Hello World?**

Yes, a few. Take a look at the source code of this documentation, for example.

- **I have an `@1` in the output where an argument substitution should take place.**

This usually has one of the following two causes:

1. You forgot the asterisk in the language macro.
2. You did strange things with catcodes. In most cases, people forgot to cancel a `\makeatletter` within the document environment with a `\makeatother`.

- **Can I omit an `@n` parameter in the starred version for one language?**

No. Besides, then the argument would obviously no longer be shared.

- **Can I use `@n` parameters in mixed order or more than once?**

Yes.

- **I want to use `@` not as a parameter in a star variant of the language macro.**

Just do it.

- **But I need `@42`.**

Use `@{ }42`

---

12. on CTAN as `csquotes`: <http://mirrors.ctan.org/macros/latex/contrib/csquotes/>

- **I use Russian, but the font is incorrect.**

This is not a problem with `langselect` (which does not deal with the font at all), but probably with Babel, `fontenc`, `fontspec`, etc. But maybe you find the hooks useful, cf. section 4.1.

- **My question does not appear here.**

You may ask at `langselect/language/after` <https://github.com/werner-matthias/osglecture/issues>

## 6 Implementation

```
1 \NeedsTeXFormat{LaTeX2e}[2022/06/01]
2 \ProvidesExplPackage{\packagename}{\packagedate}{\packageversion}{support for different
3 languages versions from a common source}
```

`\olsIsoTags` contains all valid ISO 639-1 code tags.

```
4 \seq_const_from_clist:cn{\olsIsoTags}{aa,ab,ae,af,ak,am,an,ar,as,av,ay,az,ba,be,
5   bg,bh,bi,bm,bn,bo,br,bs,ca,ce,ch,co,cr,cs,cu,cv,cy,da,de,dv,dz,ee,el,en,eo,es,
6   et,eu,fa,ff,fi,fj,fo,fr,fy,ga,gd,gl,gn,gu,gv,ha,he,hi,ho,hr,ht,hu,hy,hz,ia,id,
7   ie,ig,ii,ik,io,is,it,iu,ja,jv,ka,kg,ki,kj,kl,km,kn,ko,kr,ks,ku,kv,kw,ky,la,
8   lb,lg,li,ln,lo,lt,lu,lv,mg,mh,mi,mk,ml,mn,mr,ms,mt,my,na,nb,nd,ne,ng,nl,nn,no,
9   nr,nv,ny,oc,oj,om,or,os,pa,pi,pl,ps,pt,qu,rm,rn,ro,ru,rw,sa,sc,sd,se,sg,si,sk,
10  sl,sm,sn,so,sq,sr,ss,st,su,sv,sw,ta,te,tg,th,ti,tk,tl,tn,to,tr,ts,tt,tw,ty,ug,
11  uk,ur,uz,ve,vi,vo,wa,wo,xh,yi,yo,za,zh,zu}
12
13 \newcommand\IfTagIsValidF[2]{%
14   \seq_if_in:ceF{\olsIsoTags}{#1}{#2}
15 }
16 \RequirePackage{varsfromjobname}[2025/08/03]
```

The results of `\varsfromjobname` and `\str_range:Nnn` have catcode 12, but ISO tags and options have catcode 11. We standardize using a wrapper on catcode 11.

```
17 \newcommand\olsChangeCatEleven[1]{
18   \def\tmpa{#1}
19   \edef\tmpb{
20     \noexpand\scantokens{
21       \noexpand\edef\noexpand\olsCatElven{\tmpa}
22     }
23   }
24   \tmpb
25 }
```

If necessary, we map language codes to language variants. When interacting with external code, the language variant is then passed on.

```

26 \cs_new:Npn \ols_use_map:n #1 {
27   \cs_if_exist_use:cF { olslangmap / #1 }{ #1 }
28 }

```

We define some options. All except `languages` are optional. We will store some results in variables that we first create.

```

29 \bool_new:N\ols_load_babel
30 \bool_new:N\ols_load_polyglossia
31 \DeclareKeys{
32   languages.clist_gset:N = \ols_langs,
33   languages.usage=load,
34   targetlang.code = {

```

The option will be only evaluated in case, the target language isn't already set by definition of `langselect/language/aftertargetlang\olsTargetLanguage`.

```

35   \cs_if_exist:NF\olsTargetLanguage{

```

Is an element of the jobname given?

```

36   \bool_if:nT{
37     \str_if_eq_p:ee{\str_range:nnn{#1}{1}{3}}{job}
38   }{%
39     \olsChangeCatEleven{\getfromjobname{\str_range:Nnn{#1}{-1}{-1}}}
40     \xdef\olsTargetLanguage{\olsCatElven}
41   }

```

Should the target language be derived from `r langselect/language/afterbabelpolyglossia`?

```

42   \bool_if:nT{
43     \str_if_eq_p:nn{#1}{babel} || \str_if_eq_p:nn{#1}{polyglossia}
44   }{
45     \str_if_empty:eF{\BCPdata{main.language}}{
46       \edef\olsTargetLanguage{\BCPdata{main.language}}
47     }
48   }

```

Should the language of the meta data be used?

```

49   \str_if_eq:nnTF{#1}{meta}{
50     \IfDocumentMetadataTF{
51       \edef\tmpa{\GetDocumentProperty{document/lang}}
52       \edef\tmpb{\str_range:Nnn\tmpa{1}{2}}
53       \olsChangeCatEleven{\tmpb}
54       \edef\olsTargetLanguage{\olsCatElven}
55     }{
56       \PackageWarningNoLine{\packagename}{No~meta~data~provided}

```

## 6 Implementation

```
57     }
58   }
59 }
```

If no language is assigned by now, the argument is assumed to be the language. We check later, whether it is valid.

```
60   \cs_if_exist:NF \olsTargetLanguage {
61     \edef\olsTargetLanguage{#1}
62   }
63 },
```

Prefix of the user language commands.

```
64   targetlang.usage=load,
65   prefix.store=\ols_sprefix,
66   prefix.initial:n=l,
67   prefix.usage=load,
```

Automatic generation of the language commands?

```
68   auto.bool_set:N=\ols_generate,
69   auto.usage=load,
70   auto.initial:n=true,
```

Automatic removal of heading and trailing spaces?

```
71   trim.bool_set:N=\ols_trim,
72   trim.usage=load,
73   trim.initial:n=true,
```

Mapping to language variants.

```
74   map.code:n = {
75     \prop_new:N \ols_lang_pl
76     \prop_clear:N \ols_lang_pl
77     \prop_set_from_keyval:Nn \ols_lang_pl {#1}
```

Since property lists cannot be expanded, we convert the entries to ordinary macros.

```
78   \prop_map_inline:Nn \ols_lang_pl
79     { \cs_gset:cpn { olslangmap / \tl_to_str:n {##1} } {##2} }
80   },
```

Load @1 or @2 with optionsbabelpolyglossia.

```
81   load~babel .code:n = {
82     \bool_set_true:N \ols_load_babel
83     \PassOptionsToPackage{#1}{babel}
```

## 6 Implementation

```

84 },
85 load~polyglossia .code:n = {
86   \bool_set_true:N \ols_load_polyglossia
87   \PassOptionsToPackage{#1}{polyglossia}
88 },
89 load.usage = load,
90 unified~shorthands.bool_set:N=\ols_shorthands,
91 unified~shorthands.initial:n=true,
92 unified~shorthands.usage=load
93 }%
94 \ProcessKeyOptions\relax

```

If no target language has been assigned via `\olsTargetLanguage` or `targetlang`, we check again whether a language can be identified via language packages or `\DocumentMetadata`.

```

95 \cs_if_exist:NF \olsTargetLanguage {
96   \str_if_eq:eeF{\BCPdata{main.language}}{\edef\olsTargetLanguage{\BCPdata{main.language}}}
97 }
98 \cs_if_exist:NF \olsTargetLanguage {
99   \IfDocumentMetadataT{
100     \edef\tmpa{\GetDocumentProperty{document/lang}}
101     \edef\tmpb{\str_range:Nnn\tmpa{1}{2}}
102     \olsChangeCatEleven{\tmpb}
103     \edef\olsTargetLanguage{\olsCatElven}
104   }
105 }
106 \cs_if_exist:NF \olsTargetLanguage {
107   \PackageWarningNoLine{\packagename}{Can't~identify~any~target~language.
108     \MessageBreak Falling~back~to~'en'}
109   \def\olsTargetLanguage{en}
110 }

```

We are doing some checks. First, we check whether a valid target language could be determined.

```

111 \IfTagIsValidF{\olsTargetLanguage}{
112   \PackageWarningNoLine{\packagename}{Do~not~recognize~language~'\olsTargetLanguage'.
113     \MessageBreak Take~'en'~as~replacement}
114   \def\olsTargetLanguage{en}
115 }

```

Are at least two selectable languages specified?

```

116 \int_compare:nNt{\clist_count:N{\ols_langs}} < {2}{
117   \PackageError{\packagename}{No~sufficient~number~of~selectable
118     \MessageBreak languages~provided}{
119     You~have~to~provide~a~list~of~two~or~three\MessageBreak

```

```

120     languages~via~'language'~option.}
121   \aftergroup\endinput
122 }

```

Currently, `langselect` can't process more than three languages. Thus, we issue a warning if the list of selectable languages contains more than three languages.

```

123 \int_compare:nNt{\clist_count:N\ols_langs} > {3}{
124   \PackageWarningNoLine{\packagename}{Too~many~selectable~languages~provided.
125     \MessageBreak I~will~ignore~the~superflous~languages}
126 }

```

Next, we test the validity of the selectable languages. They should form ISO 639-1 codes.

```

127 \clist_map_inline:Nn\ols_langs {
128   \IfTagIsValidF{#1}{
129     \PackageError{\packagename}{Couldn't~resolve~selectable~language~'#1'}{
130       Use~valid~ISO-639-1~code~in~option~'languages'.}
131   \aftergroup\endinput
132 }
133 }

```

The list of selectable languages should include the target language.

```

134 \clist_if_in:NVF\ols_langs{\olsTargetLanguage}{
135   \PackageError{\packagename}{Target~language~'\olsTargetLanguage'~is~not~\MessageBreak
136     in~the~list~of~selectable~languages}{
137     Check~option~'language'~and~'targetlang'~.}
138 }

```

Babel and Polyglossia cannot both be loaded.

```

139 \bool_if:nT{
140   \ols_load_babel && \ols_load_polyglossia
141 }{
142   \bool_set_false:N \ols_load_babel
143   \bool_set_false:N \ols_load_polyglossia
144   \PackageError{\packagename}{The~options~'load~babel'~and~'load~polyglossia'
145     \MessageBreak are~mutual~exclusive,~you~can't~use~both}{Use~one~load~option,~only.}
146 }
147 \bool_if:nT{
148   ( \cs_if_exist_p:c{ver@babel.sty} || \cs_if_exist_p:c{ver@polyglossia.sty} )
149   &&
150   ( \ols_load_babel && \ols_load_polyglossia )
151 }{
152   \bool_set_false:N \ols_load_babel
153   \bool_set_false:N \ols_load_polyglossia

```

## 6 Implementation

```

154 \PackageWarningNoLine{\packagename}{Can't~load~babel/polyglossia~for
155 ~you,since\MessageBreak
156 a~language~package~is~already~loaded.\MessageBreak
157 I'll~ignore~that~option}
158 }

```

s the name of the target language as it is passed on to external code, if applicable, i.e. `\DocumentMetadata`, language packs, and hooks.`langselect/language/after\ols_mapped_target_lang`

```

159 \edef\ols_mapped_target_lang{\ols_use_map:n{\olsTargetLanguage}}

```

The target language should now be determined. We write the information into the log.

```

160 \PackageNoteNoLine{\packagename}{Target~language~is~'\olsTargetLanguage'~(\ols_mapped_target_lang)}

```

If needed, we correct the language metadata set by `\DocumentMetadata`. Now, Babel (for example) does not derive a wrong main language.

```

161 \IfDocumentMetadataT{
162   \edef\olsDocLang{\exp_args:Ne\str_range:nnn{\GetDocumentProperty{document/lang}}{1}{2}}
163   \bool_if:nF{
164     \str_if_eq_p:ee{\olsDocLang}{\olsTargetLanguage}
165   }{
166     \PackageWarningNoLine{\packagename}{Target~language~'\olsTargetLanguage'~
167     doesn't~comply~with\MessageBreak \c_backslash_str
168     DocumentMetadata~'\GetDocumentProperty{document/lang}'.\MessageBreak
169     I'll~try~to~overwrite~the~meta~data}
170     \DocumentMetadata{lang=\ols_mapped_target_lang}
171   }
172 }

```

Since the name of the language macro is unknown, patching or generic hooks are more complicated. We therefore provide our own hooks.

```

173 \NewHookWithArguments{langselect/language/before}{1}
174 \NewReversedHook{langselect/language/after}
175 \NewHookWithArguments{langselect/argument}{2}

```

A Boolean to indicate whether more than two selectable languages have been specified.

```

176 \bool_new:c{ols_trilang}
177 \int_compare:nNnTF{\clist_count:N{\ols_langs}} > {2}{
178   \bool_set_true:c{ols_trilang}
179 }{
180   \bool_set_false:c{ols_trilang}
181 }
182 \def\olsFrstLanguage{\clist_item:Nn \ols_langs{1}}
183 \def\olsScndLanguage{\clist_item:Nn \ols_langs{2}}

```

## 6 Implementation

```

184 \bool_if:cT{ols_trilang}{
185   \def\olsThrdLanguage{\clist_item:Nn \ols_langs{3}}
186 }
187 \bool_if:NTF\ols_trim{
188   \let\ols_trim:n=\tl_trim_spaces:n
189 }{
190   \let\ols_trim:n=\relax
191 }

```

The commands `\olsMakeBilingualMacro` and `\olsMakeTrilingualMacro`, resp., are used to generate the actual language commands.

```

192 % #1: prefix, #2,#3: selectable languages, #4: base name
193 \NewDocumentCommand{\olsMakeBilingualMacro}{0{\ols_sprefix} m m o}{
194   \str_if_eq:eeT{\olsTargetLanguage}{#2} {
195     \IfValueTF{#4}{
196       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#4}{s +m +m}{
197         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
198         \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##2}}}{\ols_trim:n{##2}}
199         \UseHook{langselect/language/after}
200       }
201     }{
202       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3}{s +m +m}{
203         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
204         \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##2}}}{\ols_trim:n{##2}}
205         \UseHook{langselect/language/after}
206       }
207     }
208   }
209   \str_if_eq:eeT{\olsTargetLanguage}{#3}{
210     \IfValueTF{#4}{
211       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#4}{s +m +m}{
212         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
213         \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##3}}}{\ols_trim:n{##3}}
214         \UseHook{langselect/language/after}
215       }
216     }{
217       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3}{s +m +m}{
218         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
219         \IfBooleanTF{##1}{\ols_trim:n{\olsProcessArgs{##3}}}{\ols_trim:n{##3}}
220         \UseHook{langselect/language/after}
221       }
222     }
223   }
224 }

```



## 6 Implementation

Now do the same thing again for *three* languages.

```

225 % #1: prefix, #2...#4: selectable languages, #5: base name
226 \NewDocumentCommand{\olsMakeTrilingualMacro}{0{\ols_sprefix} m m m o}{
227   \str_if_eq:eeT{\olsTargetLanguage}{#2} {
228     \IfValueTF{#4}{
229       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#5}{s +m +m +m}{
230         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
231         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##2}}
232         \UseHook{langselect/language/after}
233       }
234     }{
235       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3#4}{s +m +m +m}{
236         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
237         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##2}}
238         \UseHook{langselect/language/after}
239       }
240     }
241   }
242   \str_if_eq:eeT{\olsTargetLanguage}{#3} {
243     \IfValueTF{#4}{
244       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#5}{s +m +m +m}{
245         \UseHook{langselect/language/before}{\ols_mapped_target_lang}
246         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##3}}
247         \UseHook{langselect/language/after}
248       }
249     }{
250       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3#4}{s +m +m +m}{
251         \UseHook{langselect/language}{\ols_mapped_target_lang}
252         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##3}}
253         \UseHook{langselect/language/after}
254       }
255     }
256   }
257   \str_if_eq:eeT{\olsTargetLanguage}{#4} {
258     \IfValueTF{#4}{
259       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#5}{s +m +m +m}{
260         \UseHook{langselect/language}{\ols_mapped_target_lang}
261         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##4}}
262         \UseHook{langselect/language/after}
263       }
264     }{
265       \ExpandArgs{c}\NewExpandableDocumentCommand{#1#2#3#4}{s +m +m +m}{
266         \UseHook{langselect/language}{\ols_mapped_target_lang}

```

## 6 Implementation

```

267         \IfBooleanTF{##1}{\ols_trim:{\olsProcessArgs{##2}}}{\ols_trim:{##4}}
268         \UseHook{langselect/language/after}
269     }
270 }
271 }
272 }

```

If automatic generation is not disabled, the specific language commands are created. The names are  $\backslash\langle prefix\rangle\langle language_1\rangle\langle language_2\rangle$  or  $\backslash\langle prefix\rangle\langle language_1\rangle\langle language_2\rangle\langle language_3\rangle$ .

```

273 \bool_if:cTF{ols_generate}{
274     \bool_if:cTF{ols_trilang}{
275         \olsMakeTrilingualMacro{\olsFrstLanguage}{\olsScndLanguage}{\olsThrdLanguage}
276     }{
277         \olsMakeBilingualMacro{\olsFrstLanguage}{\olsScndLanguage}
278     }
279 }{
280     \PackageInfoNoLine{\packagename}{No~language~macros~generated.\MessageBreak
281     Use~\string\olsMakeBilingualMacro~or~\string\olsMakeTrilingualMacro~to~generate.}
282 }

```

For the starred variants of the language commands, we create a command that processes the subsequent arguments.

```

283
284 \seq_new:N \ols_args_seq
285 \int_new:N \ols_num_of_args
286 \tl_new:N \ols_format_str_tl
287

```

Since the placeholder character @ appears as a literal in the following code, it must have the same catcode as in the document.

```

288 \makeatother
289 \NewDocumentCommand \olsProcessArgs { +m } {
290     \tl_gset:Nn \ols_format_str_tl{#1}
291     \int_zero:N \ols_num_of_args

```

Find the highest value where @s contained in the format string. $\text{langselect/language/after}$  $n$

```

292     \int_step_inline:nnnn { 9 } { -1 } { 1 }
293     {
294         \regex_if_match:nnT {@##1}{ #1 }
295         {
296             \int_set:Nn \ols_num_of_args { ##1 } \prg_break:
297         }
298     }

```

If no placeholder character is found, output the text in original form. Otherwise, the replacement command is called.

```

299   \int_compare:nNnTF {\ols_num_of_args} = { 0 }
300   { #1 }{
301     \seq_clear:N \ols_args_seq
302     \ols_collect_replace:n \ols_num_of_args
303   }
304 }

```

receives *one* argument, namely the number of remaining replacement texts. However, it consumes *two* arguments. It obtains the second one from the token stream following the command, where it finds the next replacement text. This is stored in a token sequence and is called `again.langselect/language/after\ols_collect_replace`

```

305 \cs_new_protected:Npn \ols_collect_replace:n #1 #2{
306   \IfHookEmptyTF{langselect/argument}{
307     \seq_put_right:Nn \ols_args_seq { #2 }
308   }{
309     \seq_put_right:Nn \ols_args_seq {
310       \UseHook{langselect/argument}{\ols_mapped_target_lang}{#2}
311     }
312   }
313   \int_compare:nNnTF { #1 } = { 1 }
314   {

```

Once all replacement texts have been read in, they are applied one after the other to the placeholder expressions.

```

315   \int_step_inline:nn { \ols_num_of_args }
316   {
317     \tl_analysis_log:n{@}
318     \tl_analysis_log:n{\ols_format_str_tl}
319     \tl_replace_all:Nnn \ols_format_str_tl
320     { @##1 }
321     { \seq_item:Nn \ols_args_seq { ##1 } }
322   }
323   \ols_format_str_tl
324 }{
325   \ols_collect_replace:n { \int_eval:n { #1 - 1 } }
326 }
327 }
328 \makeatletter

```

If desired, Babel or Polyglossia is loaded and the target language is set as the main language.

## 6 Implementation

```
329 \bool_if:NT \ols_load_babel{
330   \RequirePackage[main=\ols_mapped_target_lang, provide*=*]{babel}
331   \bool_if:nT{
332     \ols_shorthands && !\str_if_eq_p:nn{\olsTargetLanguage}{de}}
333   {
334     \languageshorthands{ngerman}
335   }
336 }
337 \bool_if:NT\ols_load_polyglossia{
338   \RequirePackage{polyglossia}
339 }
```

Regardless of when a language pack was loaded, an attempt will be made to activate the target language.

```
340 \cs_if_exist:cT{ver@babel.sty}{
341   \AtBeginDocument{
342     \selectlanguage{\ols_mapped_target_lang}
343   }
344 }
345 \cs_if_exist:cT{ver@polyglossia.sty}{
346   \bool_if:NTF\ols_shorthands{
347     \setdefaultlanguage[babelshorthands=true]{\ols_mapped_target_lang}
348   }{
349     \setdefaultlanguage{\ols_mapped_target_lang}
350   }
351 }
352 \bool_if:NT\ols_shorthands {
353   \RequirePackage[autostyle=true]{csquotes}
354   \AtBeginDocument{
355     \cs_if_exist:cT{usesshorthands}{
356       \usesshorthands*{"}
357       \defineshorthand{"`"}{\openautoquote}
358       \defineshorthand{"'"}{\closeautoquote}
359       \defineshorthand{" /"}{\textormath{\bbl@allowhyphens
360         \discretionary{/}{/}{/}\bbl@allowhyphens}{}}
361     }
362   }
363 }
```

## Index

`auto`, 5, 6

`babel` (package), 11, 12

`bicaption` (package), 3

`\closeautoquote`, 9

`comment` (package), 3

`csquotes` (package), 6, 9

CTAN, 3, 4, 9

`\DocumentMetadata`, 2, 5, 13, 15

EIJKHOUT, Victor, 3

`\enquote`, 9

`fontenc` (package), 10

`fontspec` (package), 10

`\foreignlanguage`, 3, 8

GREWE, Richard, 3

`\include`, 2

`\label`, 7

`languages`, 5, 11

`load babel`, 5, 8

`load polyglossia`, 8

`\makeatletter`, 9

`\makeatother`, 9

`map`, 5

`multilang` (package), 3

`\ols_collect_replace`, 19

`\ols_mapped_target_lang`, 15

`\olsIsoTags`, 10

`\olsMakeBilingualMacro`, 8, 16

`\olsMakeTrilingualMacro`, 8, 16

`\olsTargetLanguage`, 2, 4, 5, 8, 11, 13

`\openautoquote`, 9

`osglecture` (package), 9

`polyglossia` (package), 11, 12

`prefix`, 5, 8

`\section`, 7

`\selectlanguage`, 3

`targetlang`, 4, 5, 8, 11, 13

`translations` (package), 3

`translator` (package), 3

`trim`, 5

`unified shorthands`, 6, 9

`\usepackage`, 2, 4

`\varsfromjobname`, 10

`varsfromjobname` (package), 4

`xt_capt` (package), 3